

# The Conceptual Space of Software Architecture

The Latent Variables That Generate Architectural Decisions

Edwin O Onyango

Dartmouth College

`edwin.o.onyango.jr@gmail.com`

Walter Wagunde

Tufts University

August 2026

## Abstract

What is software architecture, such that a machine could hold it? Every coding agent today navigates a codebase through its observables: the file tree, a symbol graph, text similarity, and a frozen prose file in which humans write down what actually matters. This paper asks whether the thing being written down has a small generating basis. By that we mean a short list of latent variables such that every architectural decision, in any system, is a move along one of them, the way physics replaced a bestiary of phenomena with a few forces.

We answer in three ways. The first is theoretical. We start from one premise: software is kept alive by a bounded population of agents, human or machine, while the world changes around it. From that premise we derive what an architecture must be. It is not a structure. It is a set of decisions laid over four graphs. The problem has a graph of constraints. The code has a graph of dependencies. The runtime has a graph of interactions. The organization has a graph of communication. We then read sixty years of theory, from Parnas, Dijkstra, Simon, and Conway through design structure matrices, architectural styles, and architecture as decisions, and we find the same small set of variables appearing again and again under different names. Where do the cuts fall on each graph? Do the four cuts line up with one another? What kind of interaction crosses each boundary? Which orders of visibility and abstraction sit over the cuts? And for each decision: how volatile is it, when does it take force, what would reversing it cost, and how far does it reach? One such agreement between two theories could be a coincidence. What we find is a pattern of them. The measure now called propagation cost was derived twice, once from a change-cost argument and once from a build-cost argument, by authors who did not cite each other. Binding time appears independently in five separate frameworks. The dominance order was discovered four times. When many lines of work that start from different premises and share no vocabulary keep arriving at the same variables, the most reasonable explanation is that the variables are really there. That logic of convergent evidence is the backbone of the paper.

The second way is empirical. We survey eighteen major systems and five industrial migrations, and of every architectural difference we find, we ask one simple question: is this a choice, or is it decay? Some differences are choices. Excellent teams pick opposite answers to the same question, each side for good reasons, and over the years real systems move in both directions. Other differences are not choices at all. Nobody chooses a tangle of circular dependencies; systems slide into it, and climbing out costs effort. The paper keeps these two kinds strictly apart, calling the choices coordinates and the decay directions potentials. The clearest single lesson from the migrations is also simple. How you divide your code and how you deploy it are two separate decisions, and the teams that treated them as one are the ones that later wrote post mortems about it; three of five name exactly this as the root error. The third way is computational. Roughly three quarters of the variables can be estimated today from a bare repository or its git history. Only the intent-carrying ones require a human, and for those the reflexion model loop is a proven protocol: the machine drafts, the human corrects, the machine enforces.

Finally, we assemble the basis into a conceptual space in Gärdenfors' sense, the picture

cognitive science uses for how minds hold concepts: qualities become dimensions, concepts become regions, similarity becomes nearness. Styles are its prototype regions. Tactics and the six modular operators are its move set. Quality attributes form a value field, held separate from the coordinates. The metric is asymmetric and built on reachability. That choice turns Tversky’s classical objection, that human judgments of similarity refuse to behave like distances, from a problem into a prediction. We are equally explicit about what the space is not. It is not fixed, since framing re-weights it. It is not Euclidean, since edge costs are nonlocal. It is not one partition, since recovery algorithms disagree precisely because each optimizes a different real dimension. We close with falsifiable predictions and with a construction recipe: what an agent must compute so that, on seeing files, functions, and classes, it is aware of every variable in play.

## Contents

|          |                                                                                                         |           |
|----------|---------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The question, sharpened</b>                                                                          | <b>2</b>  |
| <b>2</b> | <b>Method: three triangulating tests</b>                                                                | <b>3</b>  |
| 2.1      | The convergence test . . . . .                                                                          | 3         |
| 2.2      | The two-way traffic test . . . . .                                                                      | 4         |
| 2.3      | The computability test . . . . .                                                                        | 4         |
| 2.4      | What we deliberately keep separate . . . . .                                                            | 4         |
| <b>3</b> | <b>The object: what an architecture is</b>                                                              | <b>5</b>  |
| 3.1      | Boundedness forces cuts; change decides where they go . . . . .                                         | 5         |
| 3.2      | The four graphs . . . . .                                                                               | 5         |
| 3.3      | Decisions, not structures, are the substance . . . . .                                                  | 6         |
| <b>4</b> | <b>The basis, tier by tier</b>                                                                          | <b>7</b>  |
| 4.1      | Tier I: boundary geometry . . . . .                                                                     | 7         |
| 4.2      | Tier II: order structure . . . . .                                                                      | 8         |
| 4.3      | Tier III: the commitment calculus . . . . .                                                             | 9         |
| 4.4      | Tier IV: the value field and the dynamics . . . . .                                                     | 10        |
| 4.5      | The basis at a glance . . . . .                                                                         | 11        |
| <b>5</b> | <b>The space: geometry, prototypes, moves, metric</b>                                                   | <b>12</b> |
| 5.1      | Coordinates and locality . . . . .                                                                      | 12        |
| 5.2      | Styles and patterns are the prototype system . . . . .                                                  | 13        |
| 5.3      | The move set . . . . .                                                                                  | 13        |
| 5.4      | The metric: asymmetric, use-shaped, and answerable to Tversky . . . . .                                 | 14        |
| <b>6</b> | <b>Reduction tests: the familiar vocabulary is composite</b>                                            | <b>14</b> |
| 6.1      | “Microservices” is four coordinates, not one . . . . .                                                  | 14        |
| 6.2      | “Layering” is three partial orders . . . . .                                                            | 15        |
| 6.3      | “Coupling” is a product, plus a residual, plus a ghost . . . . .                                        | 15        |
| 6.4      | “Hard to change” is a computable product . . . . .                                                      | 15        |
| 6.5      | “The architecture” recovered from code is a projection, and the recovery literature proves it . . . . . | 15        |
| 6.6      | “Separation of concerns” and the tyranny of one cut . . . . .                                           | 16        |
| <b>7</b> | <b>The empirical shape of the space</b>                                                                 | <b>16</b> |

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| 7.1       | The decision points localize the dimensions . . . . .     | 16        |
| 7.2       | Two-way traffic identifies the free coordinates . . . . . | 16        |
| 7.3       | What varies together, and what refuses to . . . . .       | 17        |
| <b>8</b>  | <b>What a machine can measure</b>                         | <b>17</b> |
| 8.1       | The evidence ladder . . . . .                             | 17        |
| 8.2       | The gap, and the tell . . . . .                           | 17        |
| 8.3       | Why the decision store becomes viable now . . . . .       | 18        |
| <b>9</b>  | <b>Honest limits and open problems</b>                    | <b>18</b> |
| <b>10</b> | <b>Falsifiable predictions</b>                            | <b>19</b> |
| <b>11</b> | <b>Coda: building the space</b>                           | <b>19</b> |

---

## 1 The question, sharpened

We want to begin somewhere that almost feels too obvious to mention, because it is usually in those places that the most important assumptions sit unnoticed.

Ask two engineers to draw the architecture of the system they share and you will get two different diagrams, drawn at different granularities, organized around different concerns. Now ask those same two engineers which proposed change is dangerous, which module is load-bearing, and which shortcut will be regretted, and they will largely agree. Whatever the architecture is, it is not the diagrams. It is the thing that lets two people who have never compared diagrams make the same predictions. This paper is an attempt to pin that thing down, precisely enough that a machine could estimate it.

There is a weak version of this research and a strong one, and only the strong one is worth doing.

The weak version says that architectures differ in many ways, and that we should catalog the ways: styles, patterns, tactics, smells. This has been done, repeatedly and well [38, 98, 43], and it is nearly useless for our purpose, because a catalog has no generative power. It tells you what has been seen, not what must be. A catalog cannot tell an agent, facing a system it has never seen, which questions to ask of it.

The strong version is a claim about generating variables. By latent we mean nothing exotic, only the statistician’s plain sense of the word: quantities never observed directly, whose existence is earned by how much of the observable record they explain. The claim is that there exists a small basis  $B$  of such quantities, and that every architectural decision, from “should this be two services or one” down to “should this function know about that type,” is a move along the elements of  $B$ . Every architecture is then a point positioned by them, or more precisely, as we will see, a trajectory. If  $B$  exists and is small, then understanding a codebase is not memorizing its shape. It is estimating its coordinates. “Know one system, understand many” stops being an aspiration and becomes a structural consequence, because an observation about one boundary updates a shared latent and moves expectations about every other boundary.

This is the same move made twice before in adjacent territory. Physics replaced phenomena with forces. Machine learning replaced hand-written rules with latent representations that earn their existence by making prediction easier. The object here is different, and choosing it correctly is most of the work. We are not asking what architecture documentation contains; that question

produces templates. We are not asking what architecture recovery outputs; that question produces partitions. We are asking: **what are the degrees of freedom of the design, such that fixing their values determines every structural fact about a system that an engineer would call architectural, and such that every architectural disagreement is a disagreement about their values?**

The question has an unusual property. It has been answered many times, partially, by people who mostly did not read each other. Parnas answered it with the distribution of anticipated change [1]. Dijkstra answered it with ordered abstraction [5]. Simon answered it with interaction density [7]. Conway answered it with the shape of the designing organization [9]. Baldwin and Clark answered it with option value over a dominance hierarchy [15]. Shaw and Clements answered it with an explicit dimension table [39]. Fielding answered it by deriving an architecture as a walk through constraint space [42]. Jansen and Bosch answered it by declaring the decisions themselves to be the architecture [53]. The scandal of the field is not that no theory exists. It is that eight partial theories exist, and nobody has computed their union, checked it against the decision points of real systems, and asked what is irreducible.

That is this paper’s job. Its thesis, stated now and defended throughout:

**Claim 1.** *Software architecture is a decision structure laid over four graphs. Its latent variables fall into four tiers: (i) **boundary geometry**, which says where the cuts fall on each graph, how the four cuts align, and what kind of interaction crosses each boundary; (ii) **order structure**, the partial orders of use, abstraction, and dominance laid over the cuts; (iii) **a commitment calculus**, which attaches to each decision its volatility, binding time, reversal cost, blast radius, and commitment state; and (iv) **a value field and a dynamics**, quality-attribute gradients held separate from the coordinates, and an entropic drift that makes architecture a trajectory rather than a point. These tiers are mutually irreducible, jointly generative, and largely machine-estimable.*

The rest of the paper earns each clause of that sentence.

## 2 Method: three triangulating tests

A proposed latent variable is cheap. The literature contains hundreds of candidate “dimensions of architecture,” and admitting them by assertion would only add one more catalog to the pile. We admit a variable into the basis only when it passes tests of a different kind. Three tests do the work throughout this paper.

### 2.1 The convergence test

If a quantity is a real dimension of the space rather than an artifact of one author’s formalism, it should be discovered independently, from different premises, by people solving different problems. The flagship example is the density of the transitive closure of the dependency graph. MacCormack, Rusnak and Baldwin derived it as propagation cost, the expected fraction of a system affected by a random change [19]. A decade earlier, Lakos had derived it as average cumulative component dependency, the expected cost of an incremental build and test [29]. Same mathematical object, reached once from a change-cost argument and once from a build-cost argument. When two theories that share no citations and no vocabulary force the same coordinate, the coordinate is probably in the territory, not the map. And one such convergence could still be a coincidence, which is why the method is cumulative. We collect convergences until coincidence stops being a plausible explanation. Binding time, for example, appears

independently in five separate frameworks (Axis 12), and the dominance order was discovered four times (Axis 8). We will flag every such convergence as it arrives. There are more of them than the field seems to know.

## 2.2 The two-way traffic test

Faced with any difference between two systems, we ask one simple question. Is this a choice, or is it decay? A choice is a genuine degree of freedom. Competent designers may sit at either pole, depending on the forces they face, and we will call such an axis a dimension, or a coordinate. Decay is different. It is a direction no competent designer takes on purpose, a slope that systems slide down and must spend effort to climb back up, and we will call it a potential. A catalog cannot tell these two apart. The empirical record can. If respected systems occupy opposite poles of an axis with explicit, opposite justifications, and stronger still, if documented migrations exist in both directions, then the axis is a coordinate. Chromium moved its isolation boundary finer after Spectre [100, 101]. Istio merged its control plane coarser after measuring operational cost [111]. Segment went from monolith to microservices and back to monolith [108]. Apache and nginx converged on each other’s concurrency mechanisms from opposite ends [113]. Isolation granularity, centralization, binding time, and concurrency model all carry two-way traffic. By contrast, nobody migrates toward a larger cyclic core of mutually dependent files on purpose. Dependency-graph health carries one-way intentional traffic only, and it therefore enters the theory not as a coordinate but as a potential, a gradient that the dynamics descend or resist (§4.4). This distinction between coordinates and potentials, which the two-way test operationalizes, is one of the paper’s load-bearing contributions.

## 2.3 The computability test

The end state of this research program is an agent that, on opening a repository, estimates the variables. A latent variable that is well defined but unestimable in principle, whether from code, history, traces, or a bounded human interaction, can still constrain theory, but it cannot enter the working basis. We therefore tag every variable with its evidence requirement: *bare repo*, *plus git history*, *plus runtime traces*, or *plus human intent*. And we take seriously the reflexion-model result that the human-intent inputs are bounded and cheap. One engineer reflexion-modeled the 1.2 MLOC Excel codebase in about a month, iterating a hypothesized module map against computed divergences [83]. The machine drafts, the human corrects, the machine enforces. Intent is an input, not an excuse.

## 2.4 What we deliberately keep separate

Three separations are enforced everywhere, because their conflation is the commonest failure of prior syntheses. The first is coordinates versus value field. Where a system *is*, its structural position, is one thing. How *good* that position is for given goals is another. Lane’s design spaces [40], ATAM’s sensitivity points [44], and Fielding’s property table [42] are all maps between these two objects, and they lose their content if the objects are merged. The second is coordinates versus dynamics. The drift of structure under change, Lehman’s laws [12], is a vector field over the space, not an axis of it. The third is the four graphs themselves. Requirements, code, runtime, and organization each carry their own cut, and much of what the field argues about is secretly a claim about the congruence of two of them (§3.2).

### 3 The object: what an architecture is

We now derive the object whose space we are charting, from premises weak enough to be undeniable.

**Premise.** A software system is an artifact maintained by a bounded population of agents, human or machine, under sustained and partially unanticipated change, for purposes located outside the artifact.

Everything follows from the three pressures in that sentence: *boundedness*, since no agent holds the whole; *change*, since the artifact is never done, which is Lehman’s E-type condition [12]; and *external purpose*, since validity is judged in a moving world rather than against a frozen specification.

#### 3.1 Boundedness forces cuts; change decides where they go

Because no agent holds the whole, the system must be cut into parts small enough to hold. This is Simon’s argument that complex systems built by bounded processes are hierarchies of nearly decomposable parts, with interaction density high within parts and low between them [7, 8]. Throughout this paper a *cut* is nothing exotic. It is a partition of a graph’s vertices into blocks, a decision about where the lines get drawn.

But density of *which* interactions? Here sits the first deep fork in the classical literature, and it is worth slowing down for, because it is not a disagreement to be resolved. It is a pair of coordinates to be kept distinct. Structured design cuts the graph of data-flow connections [11]. Alexander cuts the graph of requirement interactions [13]. Simon cuts the graph of runtime interactions. Parnas rejected all flowchart-shaped criteria and cut by anticipated change: begin “with a list of difficult design decisions or design decisions which are likely to change,” and make each one the hidden secret of one module [1]. Interaction minimization and change localization coincide only under an assumption that nobody in the classical corpus states or tests: that change propagates along interaction edges with uniform probability. The KWIC construction is precisely a system where they disagree. Parnas’s Decomposition 1 scores tolerably on coupling and fails catastrophically on change containment. *The volatility field over decisions is therefore an independent input, not derivable from any interaction graph* (§4.3).

#### 3.2 The four graphs

Collecting the graphs that the classical theories partition, we get exactly four, and the list appears to be complete:

1.  $G_P$ , the **problem graph**: requirement and misfit variables and their interactions (Alexander’s constraint graph [13]; feature models; the domain’s own conceptual joints, which domain-driven design later named bounded contexts [49]).
2.  $G_C$ , the **code graph**: source units and their syntactic dependencies (the DSM substrate [18, 19]; the uses relation [3]).
3.  $G_R$ , the **runtime graph**: executing elements and their interactions (Fielding’s components, connectors, and data [42]; process and deployment topology).
4.  $G_O$ , the **organization graph**: the agents and their communication structure (Conway [9]).

Each classical theory is, at bottom, the same mathematical move applied to a different member of this family: partition a weighted graph so that heavy edges fall inside blocks. The moment



the family is seen whole, a new class of variables appears that no single-graph theory can express. These are the **congruence terms**, the degree of alignment between cuts on different graphs. At least three are independently load-bearing:

- $\kappa_{OC}$ , the congruence of organization and code, which is Conway alignment. This is the best-evidenced variable in the entire literature. Organizational metrics predicted failure-prone Vista binaries better than any code metric [22]. In five matched pairs of functionally equivalent products, the more distributed organization produced the more modular codebase in every pair [20]. The concentration of ownership at a boundary predicts its failure rate [24]. And loosely coupled architecture, operationalized as teams able to test and deploy without coordination, is among the strongest population-scale predictors of delivery performance [25]. The mirroring is a prevalent pattern, not a law. It weakens in open source [17], which is exactly what makes it a *variable*.
- $\kappa_{PC}$ , the congruence of problem and code: whether module boundaries track the problem's own constraint clusters. This was Alexander's original program [13], and DDD's bounded contexts are its modern operationalization [49].
- $\kappa_{CR}$ , the congruence of the logical and the physical: whether code-level module boundaries and runtime deployment boundaries coincide. The empirical record of §7 shows this is the single most expensive conflation in industrial practice. Modularity forces, meaning ownership, comprehension, and change isolation, act on the logical cut. Operational forces, meaning cost, latency, consistency, and blast radius, act on the physical cut. Three of five major migration post-mortems name their root error as having coupled the two [107, 109, 110].

### 3.3 Decisions, not structures, are the substance

A partition of four graphs is still a static description. Two further classical results force the final reconceptualization. Parnas's program-families paper models development as a tree whose nodes are incomplete designs and whose edges are design decisions. An architecture is a node, a prefix-closed set of commitments, and a family is a subtree [2]. Thirty years later, and apparently without genealogical connection, Jansen and Bosch re-derived the same identification from maintenance economics. Structure is the residue. The decisions, with their rules and constraints, are the architecture, and their loss, which they called knowledge vaporization, is the causal mechanism of design erosion [53, 54]. Taylor, Medvidovic and Dashofy made it the textbook definition: architecture is the set of principal design decisions [46]. The convergence test passes. *The point of the space is a decision set, and the structural facts, the cuts and the orders, are its shadow on the four graphs.*

This identification does real work. It explains why architecture cannot be fully recovered from code: bans and rejected alternatives leave no residue, which is why Kruchten's ontology gives ban decisions, crucial and invisible, a first-class place [55]. It explains why erosion is path dependence plus amnesia rather than bad decisions [54]. And it explains why the space's metric must be asymmetric (§5.4): moving between two architectures is not a distance between shapes but a re-decision cost, and re-deciding is directional.

**Definition 1** (Architecture). *An architecture is a pair  $(D, \sigma)$  where  $D$  is a set of design decisions carrying the commitment calculus of §4.3 and partially ordered by their restriction relationships, and  $\sigma$  is the induced structure: the cuts  $\Pi_P, \Pi_C, \Pi_R, \Pi_O$  on the four graphs, the congruences  $\kappa$  among them, the partial orders of §4.2 over the cuts, and the interaction type at each boundary (Axis 4). A codebase realizes an architecture with a conformance residual (reflexion's convergences, divergences, and absences [83]; erosion and drift are the residual's named pathologies [37]).*

## 4 The basis, tier by tier

We now assemble the basis. Each variable is stated as an axis: what varies, its formal structure, the independent discoveries that force it, which is the convergence test at work, and its evidence requirement, which is the computability test at work. The four tiers of Claim 1 organize the presentation.

### 4.1 Tier I: boundary geometry

The first tier answers the plainest questions about a system. Where are the lines drawn, on which graph, and at what scale? Do the lines on different graphs agree? And what actually crosses them?

**Axis 1** (Cut placement, per graph). For each graph  $G_i \in \{G_P, G_C, G_R, G_O\}$ : the partition  $\Pi_i$  of its vertices, with Simon’s near-decomposability ratio  $\varepsilon_i$ , the ratio of inter-block to intra-block interaction density, as its principal scalar projection. *Formal structure*: the lattice of partitions; nearly block-diagonal matrices, with a proved timescale-separation theorem for the linear case [8]. *Convergences*: Simon on runtime interactions; Alexander on requirement interactions, with an explicit min-cut algorithm [13]; structured design on data-flow connections [11]; community detection and every software-clustering objective since [33]. *Evidence*: bare repo for  $G_C$ ; traces for  $G_R$ ; authorship and communication records for  $G_O$ ; requirements analysis or domain modeling for  $G_P$ .

**Axis 2** (Granularity, per graph). The characteristic scale of the cut: statement to function to class to module to service to system on  $G_C$ ; thread to process to sandboxed process to container to node on  $G_R$ ; individual to team to department on  $G_O$ . Categorization research predicts, and practitioner discourse is consistent with, a paradigm-relative *basic level* at which practitioners reason by default: the module in monolith cultures, the service in distributed ones. Granularity is therefore not only a structural choice. It sets the resolution of the community’s default vocabulary [76, 67]. *Two-way traffic*: Chromium moved isolation finer, first process-per-site-instance [100], then strict site isolation after Spectre changed the threat model [101]; Istio consolidated four control-plane deployables into one [111]. Both were competent; the forces differed. *Evidence*: bare repo and deployment manifests.

**Axis 3** (Congruence terms).  $\kappa_{OC}, \kappa_{PC}, \kappa_{CR}$  as introduced in §3.2: matrix congruence statistics between cuts on different graphs. These are second-order variables, properties of *pairs* of cuts, and they are where several of the field’s most robust empirical results actually live [22, 20, 23, 25]. *Evidence*:  $\kappa_{OC}$  from git authorship crossed with dependency structure;  $\kappa_{CR}$  from build and deploy manifests crossed with module structure.

**Axis 4** (Boundary interaction type). At every cut edge, the interaction is not a scalar “coupling strength.” It is a structured product of six factors, enumerated independently by the boxology program [39] and the connector taxonomy [47]. The first factor is *control topology* crossed with *data topology*: each may be linear, acyclic, hierarchical, star, or arbitrary, they are stated separately, and a relation records whether they coincide and which way they point. The second is *synchronicity*: lockstep, synchronous, asynchronous, or opportunistic. The third is *data mode*: passed, shared, copy-out-copy-in, or broadcast. The remaining three are *delivery guarantees*, *cardinality*, and *state locus*, meaning where conversation state lives: in a server session, held by the client, distributed, or nowhere [42].<sup>1</sup> The classical coupling ladder (content, common,

<sup>1</sup>A sourcing caveat applies to the enumerated value sets of the boxology, Lane, connector-taxonomy, and tactics tables throughout this paper: the primary technical reports sit behind access restrictions, and the value sets are reconstructed from the published texts and secondary quotations. The dimensional *structure* is not in doubt; individual value lists remain to be verified against the primary tables.



external, control, stamp, data, in increasing order of virtue, assembled from ingredients in [11] and codified by the structured-design school [103]) is an ordinal compression of this product: historically first, dimensionally coarse. *Evidence*: bare repo for interface signatures and connector idioms; traces for the realized subset.

**Axis 5** (Interface thinness and uniformity). Per boundary: how much of the hidden side is deducible through it. This is Parnas’s “reveal as little as possible” [1], with three independently derived formalizations. Simon argued that only slow, aggregate quantities should cross a near-decomposable boundary [7]. Baldwin’s thin crossing points locate transactions where transfers are few and standardizable [16]. The information-theoretic reading counts the bits of the secret leaked by the interface [31]. To thinness add *uniformity*: bespoke versus domain-standard versus universal, as in Unix pipes and HTTP verbs, which Fielding shows trades efficiency for evolvability and visibility [42]. And add *stability asymmetry*: a system may expose stable narrow seams over volatile interiors, as in Git’s plumbing and porcelain, or Linux’s stable ABI outward and unstable API inward, making interface granularity independent of component granularity [114]. *Evidence*: bare repo.

## 4.2 Tier II: order structure

Cuts say where the lines are. The second tier says which way is up: who may know about whom, who sits above whom in interpretation, and whose decisions bind whose.

The classical corpus contains a standing warning here, first issued in Parnas’s 1972 paper itself: at least three distinct partial orders masquerade as “hierarchy,” and collapsing them is a category error [1].

**Axis 6** (The uses order). The designed relation “*A* uses *B*,” meaning the correctness of *A* requires a correct *B*, which is not the same as invokes. Its shape ranges from unconstrained digraph to strict layers. Its value is exact by construction: the downward-closed sets of the uses order are precisely the buildable, deliverable subsets of the system, so extension and contraction capability is the lattice of order ideals [3]. Simon’s stable intermediate forms, the watchmaker argument [7], are the same object seen dynamically. *Evidence*: bare repo.

**Axis 7** (The abstraction order). Dijkstra’s levels: each level implements an abstract machine above which certain resources “lose their identity.” Formally, an ordered chain of state-space quotients, whose payoff is combinatorial, since verification cost falls from the product of level state-spaces to their sum [5]. This is distinct from the uses order. A uses-hierarchy need not be a hierarchy of interpretation, and the two orders can disagree on the same system. *Evidence*: bare repo, partially; abstraction boundaries are inferable from type structure and naming, confirmable by a human.

**Axis 8** (The dominance order and design rules). Baldwin and Clark’s deepest structural claim: a modularization is not a partition alone but a partition *plus a visibility hierarchy*. Design rules are decisions fixed early and visible to all, and hidden parameters may not constrain them [15]. Cai and Sullivan showed the DSM is not primitive but the shadow of a constraint network plus this dominance order [27]. Its measurable projections are the design rule hierarchy and the decoupling level metric, evaluated on 129 projects with maintainability validation on a 41-project subset [28], though the validations to date come largely from one research group, a replication gap the theory should not paper over. Martin’s stability direction, with instability decreasing along dependency edges, and the refinement orders of interface and contract theories (alternating simulation [35]; assume-guarantee contracts [36]) are the same order structure in other clothing. Four convergent discoveries. *Evidence*: bare repo.

**Axis 9** (Dependency orientation relative to the abstraction gradient). Whether dependencies

flow toward abstraction and stability or against it. This is the main-sequence discipline, SDP and SAP [30], with hexagonal, onion, and clean architecture as its packaging [50]. On Martin’s  $(I, A)$  unit square, instability against abstractness, the discipline is literally a vector-field constraint with a named prototype region, the main sequence  $A + I = 1$ , and named pathological corners. It is the most literal small conceptual space in the existing literature, and a template for this paper’s construction. *Evidence*: bare repo, for typed languages.

**Axis 10** (Reachability density). The scalar that summarizes what the orders jointly control: the density of the transitive closure of  $G_C$ . This is propagation cost [19], and it is Lakos’s ACD/N [29], the paper’s flagship convergence. Its refinement, decoupling level, discounts legitimate dependence on design rules and can rank systems oppositely to raw propagation cost. The two are different projections, and the space needs both [28]. Derived classifications give the space some of its best-attested prototype regions: core size, core-periphery versus hierarchical versus multi-core types, element roles on the (VFI, VFO) plane. Most large real systems are core-periphery [21]. One caveat matters for the geometry: the cost of an edge is nonlocal, since a single added edge can double the transitive closure, so the space is non-Euclidean in the edge basis [29]. *Evidence*: bare repo.

### 4.3 Tier III: the commitment calculus

Tiers I and II describe the shadow. Tier III is the substance: the variables attached to the decisions themselves. The decision-centric literature supplies them, from Parnas 1976 through Jansen and Bosch, Kruchten, the rationale tradition, and the options tradition, and they compose into what we call the commitment calculus.

**Axis 11** (The volatility field). An assignment of anticipated-change weight to every design decision. This is Parnas’s generative criterion [1], factored at scale into change-source classes of hardware, behavior, and internal [4], and re-derived economically as per-module technical potential  $\sigma_i$  in the options theory, where volatility is where option value lives [15, 26]. The two sign conventions, volatility as hazard in the stability principles and volatility as resource in the options tradition, agree on the structural prescription that nothing should depend on the volatile, and disagree on valuation. The space needs the position and the sign kept separate. *Evidence*: git history, churn and co-change, as proxy; requirements volatility needs human input.

**Axis 12** (Binding time). When each decision takes force: design, compile, link, startup, or run time. This appears independently in boxology, stated separately for control and data partners [39]; in the connector taxonomy, as linkage binding [47]; in the modifiability tactics, as defer binding [43]; in program families, where family-common decisions bind early and member-variable decisions bind late [2]; and across the empirical corpus, where nginx’s compile-time modules and Eclipse’s runtime OSGi bundles were chosen oppositely under explicit forces [113, 115]. Five independent appearances. This is the strongest convergence in the paper after reachability density. *Evidence*: bare repo.

**Axis 13** (Reversal cost, blast radius, and accrued dependents). Fowler defined architecture, with a deliberate wink, as the things people perceive as hard to change, while insisting the architect’s job is to shrink that set [65]. The definition decomposes into a product of terms. The first is reversal cost: what a rewrite, a migration, or data gravity would demand. The second is blast radius: Kruchten’s scope of system, temporal, and organizational extent [55]. The third is dependents accrued, the in-degree in the decision graph, because irreversibility often arrives after the decision, by accretion. Binding time modulates all three. The options literature prices deferral against exactly these [15]. The one-way and two-way door rule keys process selection on the first term alone [66]. “Architectural” is not a category of topics. It is the high-product region of this calculus, which is why teams adopting decision records report that their hardest problem

is deciding what counts as architectural [102], and why adoption in the wild is shallow, with half of adopting repositories abandoning after fewer than five records [60]. *Evidence*: decision graph plus git history; partially inferable, confirmable by a human.

**Axis 14** (Commitment state and restriction force). Kruchten’s ordered promotion scale, from idea to tentative to decided to approved, with challenged, rejected, and obsolesced as demotions, with consistency propagation along the decision graph’s typed edges: constrains, forbids, enables, subsumes, comprises, conflicts-with, overrides, is-alternative-to [55]. And, per decision, its restriction force: how strongly its rules and constraints prune the future option space. A ban prunes maximally and leaves zero code residue [55]. A convention prunes weakly. Jansen and Bosch’s deltas make the composition explicit: an architecture at time  $t$  is a fold over the decision sequence [53]. QOC said it first and plainest: a design is a point in a space constructed by questions, which are dimensions, options, which are values, and criteria, which are the value field [56]. *Evidence*: requires the decision store, historically vaporized; see §8 for why an agent dissolves the four documented causes of that vaporization.

One honest note closes the tier. The calculus describes the decision, not the decider. The empirical decision literature adds decider variables: the thin spread of domain knowledge across a team [62]; problem structuredness selecting between rational comparison and recognition-primed deciding [63]; bounded rationality and its documented biases [64]. These govern *how* moves get made, and they mark where an agent’s estimates must substitute for records that were never going to be written.

#### 4.4 Tier IV: the value field and the dynamics

The last tier separates two things the field habitually merges with the coordinates. The first is how good a position is for a given purpose, which is a field over the space and changes when goals change. The second is which way the terrain slopes when nobody is pushing, which is a dynamics and does not care about goals at all.

**Axis 15** (The value field). Quality attributes form a field *over* the space, not axes *of* it: Fielding’s thirteen properties, ISO 25010’s eight characteristics, and the quality-attribute-scenario formulation that makes each one measurable [42, 45, 43]. Fielding’s Table 3–6 is a sampled gradient, each constraint’s signed property increments with non-additivity explicitly flagged. ATAM’s sensitivity points are parameters with large  $\partial q/\partial x$ , and its tradeoff points are parameters where two attributes’ derivatives have opposite sign [44]. A project’s criteria weights, whether the utility tree, QOC’s criteria, or MADR’s decision drivers [119], are its direction of ascent. Keeping the field separate from the coordinates is what lets goals change without remapping the terrain. And it is Lane’s structure exactly: functional dimensions from requirements, correlated by learned rules to structural dimensions of choice, the rules being the empirical map between the two spaces [40].

**Axis 16** (Entropic drift and the maintenance ratio). Lehman’s laws supply the vector field: under E-type change, structure degrades unless anti-regressive work is spent, and the controllable variable is the allocation between progressive and anti-regressive effort [12]. The empirical decay literature confirms drift as the default trajectory across hundreds of open-source versions [91], and the smell and hotspot catalogs [92, 28] name the attractor’s neighborhoods. This is where the one-way axes live. Cyclic cores, hub overloads, and modularity violations, meaning co-change without structural dependency, the misalignment of latent and manifest coupling that is measurable only from history [28], are potentials rather than coordinates. They are descended into by accident and climbed out of only by spending energy.

**Axis 17** (Framing: the attention vector). The design-cognition literature offers a hard datum.

Problem and solution spaces co-evolve [74]. The frame determines which dimensions are salient [73]. Design is constraint-dominated, with hard and soft constraints, leaky decompositions, and personalized stopping rules [72]. The space is not experienced as fixed. The defensible reconciliation, argued at length in §9, is a fixed superset of generators plus a time-varying salience weighting. Framing rotates the active subspace; it does not create axes. Gärdenfors' machinery contains exactly this knob, context-dependent dimension weights [67], and Gero's FBS ontology gives the tier structure that localizes reframing as explicit operators, reformulation types 1 through 3, between function, behaviour, and structure spaces [75].

#### 4.5 The basis at a glance

| Variable                                               | Formal structure                                                                       | Key convergences                                       | Evidence                     |
|--------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------|------------------------------|
| Cut placement $\Pi_i, \varepsilon_i$ (4 graphs)        | partition lattice; near-block-diagonal matrices                                        | Simon; Alexander; SMC; clustering objectives           | repo / traces / org / domain |
| Granularity (per graph)                                | scale ladder; basic level                                                              | Rosch; Chromium and Istio two-way traffic              | repo, manifests              |
| Congruences<br>$\kappa_{OC}, \kappa_{PC}, \kappa_{CR}$ | matrix congruence                                                                      | Conway; mirroring studies; DDD; migration post-mortems | repo + git + org             |
| Boundary interaction type                              | product: topology $\times$ sync $\times$ mode $\times$ guarantees $\times$ state locus | boxology; connector taxonomy; coupling ladder          | repo                         |
| Interface thinness, uniformity                         | bits leaked; cut bandwidth                                                             | Parnas; Simon; Baldwin; Allen and Khoshgoftaar         | repo                         |
| Uses order                                             | partial order; ideals = subsets                                                        | Parnas 1979; Simon's stable forms                      | repo                         |
| Abstraction order                                      | chain of state-space quotients                                                         | Dijkstra; refinement calculus                          | repo + human                 |
| Dominance order / design rules                         | ACN; DRH; refinement orders                                                            | Baldwin and Clark; Cai and Sullivan; Martin; contracts | repo                         |
| Dependency orientation                                 | vector-field constraint on $(I, A)$                                                    | Martin; hexagonal/onion/clean                          | repo                         |
| Reachability density                                   | transitive-closure density; DL                                                         | MacCormack $\equiv$ Lakos; Mo et al.                   | repo                         |
| Volatility field                                       | weighted decision set; $\sigma_i$                                                      | Parnas 1972; options theory                            | git + human                  |
| Binding time                                           | ordered scale per decision                                                             | 5 independent frameworks                               | repo                         |
| Reversal cost / blast radius / dependents              | product over decision graph                                                            | Fowler; Kruchten; options; Bezos                       | decisions + git              |
| Commitment state, restriction force                    | ordered states; typed decision edges; deltas                                           | Kruchten; Jansen and Bosch; QOC                        | decision store               |
| Value field, criteria weights                          | gradient field; utility tree                                                           | Fielding; ATAM; Lane; ISO 25010                        | human + measurement          |
| Entropic drift, maintenance ratio                      | vector field; potentials                                                               | Lehman; decay studies; smells                          | git                          |

| Variable                   | Formal structure                         | Key convergences                        | Evidence    |
|----------------------------|------------------------------------------|-----------------------------------------|-------------|
| Framing / attention vector | salience weighting over fixed generators | Schön; Dorst and Cross; Gärdenfors; FBS | interaction |

Seventeen entries. But the count overstates the independence, because the deep structure is the four-tier claim: cuts and their congruences; orders over the cuts; a commitment calculus over decisions; a field and a flow over the whole. Everything in the literature we surveyed, every metric, style, tactic, smell, and principle, reduces to a position, region, move, gradient, or potential expressed in these terms. §6 performs the reductions explicitly.

## 5 The space: geometry, prototypes, moves, metric

A list of variables is not yet a space. For what more is needed, we borrow from cognitive science. Gärdenfors' conceptual spaces are a simple, well-studied picture of how minds hold concepts: qualities become dimensions, concepts become regions, and similarity becomes nearness [67]. A reader meeting the idea for the first time needs only that much. The theory then specifies what must be checked rather than assumed: that the dimensions have intrinsic structure; that similarity decreases lawfully with distance [68]; that betweenness makes sense; that natural regions are convex and organized around prototypes, which is Voronoi structure; and that dimensions divide in a principled way into integral and separable bundles. We take these requirements in turn, honestly.

### 5.1 Coordinates and locality

The dimensions of §4 are of mixed type by construction. There are ordinal scales: binding time, coupling ladders, commitment states. There are continuous scalars:  $\varepsilon$ , reachability density, the congruences. There are categorical dimensions with structure: the interaction type as a product of small enumerations. And there are full partial orders: uses, abstraction, dominance. The space is therefore a hybrid, a discrete relational skeleton of graphs and orders woven with continuous coordinates of density, congruence, and semantic similarity. Betweenness holds on the continuous sub-spaces. Between synchronous and asynchronous sits deferred-synchronous, and the connector taxonomy enumerates exactly such intermediates [47]. Betweenness fails, as expected, on the discrete skeleton, where paths replace interpolation. This is not a defect. It is the same hybrid structure that navigation research already handles by treating betweenness as optional on graph-like domains [82].

Integral versus separable bundles can be read off the empirical record. The performance bundle of latency, throughput, and resource cost, and the trade surface of consistency and availability, behave as integral: one cannot be valued without valuing the others, and they combine in a Euclidean-like way in engineers' trade-off talk. The tier-one structural axes are largely separable. The paper's central empirical finding (§7) is precisely that the logical cut and the physical cut, which the field habitually fuses, vary independently.

### 5.2 Styles and patterns are the prototype system

The Gärdenfors reading of the styles literature is exact, not analogical. A style, in Fielding's definition, is a coordinated set of constraints [42]. That makes it a *region*: the set of configurations

satisfying the constraints, with the null style, the empty constraint set, as the whole space, and constraint addition as region refinement. Boxology’s tables assign each named style its coordinates on the explicit dimension set [39]. Our third research lane performs the assignment for pipes-and-filters in full: passed-stream data mode entails no shared state, which permits asynchronous incremental processing, which yields the functional-composition proof rule and the reuse properties. The region is coherent, a natural concept rather than an arbitrary conjunction, because its coordinate values are *mutually entailing*. That is what makes a style learnable from few examples and nameable in one word. It is exactly the learnability-and-communication pressure that Gärdenfors’ convexity criterion encodes [67, 71].

Patterns are prototypes at finer grain. Alexander’s own arc ran from coordinates, the 1964 misfit-graph program he later repudiated, to prototypes with forces in 1977, and the software patterns movement imported the prototype form with the forces sections intact [14, 98]. Patterns are the community’s informally collected basis vectors, each one a named equilibrium of forces with a region of variation around it, “without ever doing it the same way twice.” The recognition literature confirms that this is how experts actually operate. Recognition-primed decisions land in a prototype cell and read off the associated action [79]. Programming plans and beacons are the code-level instance [77]. And the recovered structure of real systems clusters. Most large codebases sit in the core-periphery region [21]. Martin’s main sequence is a prototype line with named pathological corners [30]. Prototype structure is not imposed by the theory. It keeps being found.

Two honest qualifications. Styles show family resemblance: no necessary-and-sufficient conditions, graded membership, “sort of microservices.” So convexity must be read as a pressure rather than a law, and genuinely disjunctive styles violate it [70]. And expert categorization is part exemplar, the war stories, “like the system at Corp X,” and part causal theory, classification by *why*. Rationale is the scarcest, most-sought information in comprehension studies [78]. A pure geometric prototype store is necessary but not sufficient. The space needs its instances and its reasons attached.

### 5.3 The move set

The space’s operations are already cataloged, at three grains, by literatures that do not cite each other:

- **Generators.** Baldwin and Clark’s six modular operators: split, substitute, augment, exclude, invert, port [15]. Every entry in the refactoring catalogs is an instance. *Invert*, which extracts a repeated function and raises it into shared infrastructure, is the origin of every platform layer.
- **Unit moves along single axes.** The tactics catalogs, several of which name their axis outright: defer binding (Axis 12); use an intermediary, which is intermediation depth; introduce concurrency, which is synchronicity; maintain multiple copies, which is replication [43].
- **Region jumps.** Style adoption as a correlated multi-dimensional displacement, with Fielding’s REST derivation as the audited exemplar: six constraint-moves from the null style, each logged with its induced property increments and costs [42].

A decision, in the commitment calculus, is a move plus a restriction. It displaces the point and prunes the reachable region through design rules and constraints [53]. The typed edges of the decision graph [55] are the algebra by which moves compose, conflict, enable, and override.



## 5.4 The metric: asymmetric, use-shaped, and answerable to Tversky

The most consequential design choice in the whole construction is the metric, and this is a point where it becomes important to slow down, because the cognitive literature issues a warning here that the architecture literature has never processed. Tversky’s classical result is that human similarity violates every metric axiom. It is asymmetric, since the variant is compared to the prototype and not conversely. It fails the triangle inequality. It is context-dependent [69]. Any theory that installs a fixed symmetric Euclidean metric over architectural descriptions is empirically false about the judgments it must model.

The escape is already present in the material. Distance in a decision space is re-decision cost: how expensive it is to move this system from here to there, under the commitment calculus. And that quantity is inherently asymmetric. Splitting a monolith and merging microservices are not inverse operations at equal price. Adding an edge is cheap and removing it is dear; the one-way axes of §4.4 are the extreme cases. It is also inherently context-dependent, since the cost depends on binding times, accrued dependents, and the current frame. The neurally grounded form of exactly this metric exists. It is the successor representation, under which nearness is expected future co-occupancy under the agent’s policy: predictive, directional, graph-friendly, and empirically supported as the hippocampal map’s actual code [80, 81]. Seed the metric with surface structure, then let realized navigation reshape it, watching which decisions actually follow which and which units actually co-change. The co-change matrix is precisely an empirical co-navigation record. Tversky’s asymmetry then stops being an objection and becomes a prediction. Architectural similarity judgments *should* be asymmetric, in the direction of re-decision cost, because the space’s true metric is reachability under work.

This yields the paper’s second structural claim:

**Claim 2.** *The conceptual space of architecture is a hybrid manifold with an asymmetric, use-shaped metric. Its distances are re-decision costs under the commitment calculus, seeded by structural similarity and reshaped by realized co-change. Its value field, held separate, orients navigation without deforming the terrain.*

## 6 Reduction tests: the familiar vocabulary is composite

A basis earns its title by rewriting the field’s vocabulary as compositions. We perform the six most load-bearing reductions. Each one dissolves a standing confusion.

### 6.1 “Microservices” is four coordinates, not one

The monolith-versus-microservices axis, as debated, conflates four different choices [51, 52]. One is deployment granularity: how many independently deployable units there are. Another is failure isolation: whether a call stays in one process or crosses a network. A third is data ownership: one transactional store, or a store per service with eventual consistency between them. The fourth is organizational alignment: whether services are sized to teams, which is the  $\kappa_{OC}$  term. The empirical record separates them cleanly. Prime Video kept its component decomposition and deleted only the network boundary [107]. Shopify added enforced module boundaries while keeping one deployable [109]. Uber added coarse logical domains over fine physical services [110]. “Modular monolith” is not a compromise. It is a coordinate assignment: logical cut fine, physical cut coarse. The debate dissolves into force-weighting on four separable axes.

## 6.2 “Layering” is three partial orders

Containment, which is Simon’s boxes-in-boxes and the module guide’s is-submodule-of [4]; use, from Parnas 1979; and abstraction, from Dijkstra 1968, are independent orders that the word “layer” merges. The classical corpus itself insists on the distinction [1]. Systems exist with clean containment and cyclic use, and with layered use that abstracts nothing. Conformance rules in practice, such as “controllers may not access repositories directly,” are constraints on the use order. C4’s levels are resolutions of the containment order [118]. Ports-and-adapters is a constraint tying the use order to the abstraction gradient (Axis 9).

## 6.3 “Coupling” is a product, plus a residual, plus a ghost

Structural coupling is the boundary interaction type (Axis 4), itself a product rather than a scalar. Logical coupling is a different measurement entirely: co-change in history. And the *misalignment* of the two, co-change without structural dependency, correlates with error-proneness and change-proneness across many projects. It is a violated information-hiding boundary leaving a historical ghost [28]. Cohesion, the traditional partner concept, may not be a dimension at all. Its metrics fail measurement-theoretic axioms and disagree with each other [32], while the information-theoretic view renders it as the residual of the cut, what remains uncompressed when the partition is chosen [31, 34]. We therefore admit coupling-type, logical coupling, and their misalignment into the basis, and demote cohesion to a derived quantity. That is a specific, falsifiable ontological claim.

## 6.4 “Hard to change” is a computable product

Fowler’s definition-with-a-wink [65] decomposes as Axis 13: reversal cost times blast radius times dependents accrued, modulated by binding time. Each term is separately estimable, and each is separately manipulable. The point of the decomposition is practical. An agent cannot act on “perceived as hard to change.” It can compute accruing dependents, measure blast radius through the visibility matrix, and flag binding times that arrived earlier than necessary.

## 6.5 “The architecture” recovered from code is a projection, and the recovery literature proves it

Recovery algorithms each optimize a formalized theory of what a module is: dependency density in Bunch’s MQ [33], recurring structural idiom in ACDC [84], concern coherence in ARC’s topic models [85], information equivalence in LIMBO [34]. Measured against engineer-certified ground truths, all score poorly, and they disagree with each other roughly as much as with the truth [86, 87]. On the single-partition theory of architecture this is an embarrassment. On the present theory it is a *measurement*. The objectives capture different real dimensions. The engineer’s decomposition integrates variables the dependency graph does not carry: ownership, deployment, domain semantics, history. Any one partition is a lossy projection of a multi-dimensional latent state, and the disagreement pattern across objectives is itself a fingerprint of where a system sits in the space.

## 6.6 “Separation of concerns” and the tyranny of one cut

A concern is a projection of the design space onto a subspace, which is Dijkstra’s own formulation [6]. The multi-view doctrine, 4+1 and the viewpoint standards, institutionalizes a small atlas of such projections keyed to stakeholder concerns [48]. The file tree realizes exactly one cut, and whatever does not align with the chosen dominant dimension becomes scattered and tangled.

This is the tyranny of the dominant decomposition [89]. Scattering is measurable, and scattered concerns carry significantly more defects [90]. The latent state therefore includes *which* dimension was chosen dominant, the concern-to-code distribution, a multi-label field over units, and the misalignment between them. None of these can be expressed by any file tree. All of them are estimable (§8).

## 7 The empirical shape of the space

Theory proposes; the deployed world disposes. Our sixth research lane surveyed eighteen major systems through their own architects' accounts, the AOSA corpus and primary engineering sources, along with seven canonical public debates, five industrial macro-migrations, and the mining-software-repositories literature. Three findings shape the theory.

### 7.1 The decision points localize the dimensions

When both sides of a real dispute articulate their forces, the disagreement names an axis. Tanenbaum and Torvalds argued isolation-boundary placement and its costs, IPC overhead against address-space protection [99]. The same axis resurfaced, thirty years later and in the participants' own terms, in Postgres's process-versus-threads deliberations [120] and in every browser's multi-process migration [100]. Hipp's "SQLite competes with fopen()" collapses a dozen surface differences into one coordinate: where the process, trust, and administration boundary sits between application and engine [104]. React Fiber's "a fiber is a reimplement of the stack" names scheduling ownership: implicit in the language runtime, or reified as data [105]. The systemd wars conflated two axes, declarative versus imperative configuration and integrated platform versus orthogonal components, and the temperature of that debate fell where the axes were distinguished [106]. In every case the engineers' own vocabulary, not the analyst's, supplies the dimension. The folk coordinates and the theoretical ones agree.

### 7.2 Two-way traffic identifies the free coordinates

Documented bidirectional migration under articulated forces is the strongest available evidence that an axis is a genuine degree of freedom. It exists for isolation granularity: Chromium finer, Istio coarser. For logical-physical congruence: Prime Video decoupled them one way, Uber the other. For concurrency model: Apache and nginx converging from opposite poles. For centralization: ZeroMQ removed the broker RabbitMQ kept; Git decentralized what SVN centralized; ZooKeeper deliberately re-centralized coordination [117]. For binding time: nginx compile-time against Eclipse hot-install. And for state representation: snapshots versus deltas versus log-as-truth, with Git famously choosing snapshots at the logical level and deltas at the physical level, splitting the axis across an abstraction boundary [114, 116, 108]. By contrast, the one-way axes, cyclic-core growth, hub overload, and scattering, attract only unintentional traffic inward and deliberate traffic outward. They are potentials, per §4.4.

### 7.3 What varies together, and what refuses to

Read the eighteen-system variation table columnwise. The columns are process model, concurrency model, extension mechanism, topology, state management, isolation boundary, binding, and synchrony. Every one of them shows multiple values across respected systems, and the columns vary with substantial independence. That is the empirical signature of separable dimensions. The strongest single regularity is negative. Attempts to force one column to determine another, deployment granularity dictating module granularity, or org structure ignored

while cutting code, are the named root causes in the migration post-mortems [107, 109, 110]. Where the population does concentrate, the concentrations are the prototype regions and role structures the geometry predicts (§5.2). Most large systems are core-periphery [21]. Production call graphs are heavy-tailed, with a few hub services carrying enormous fan-in [112]. Fan-in is power-law distributed across every code ecosystem measured [88]. This includes the hub role that pure cohesion-and-coupling theories systematically misjudge. Utilities are coupled to everyone. Quarantining them is an idiom every practitioner knows, and only the pattern-based recovery objective encodes it [84].

## 8 What a machine can measure

The computability test now pays out. We assess, variable by variable, what suffices to estimate the basis, and we measure the gap between that and what coding agents actually represent today.

### 8.1 The evidence ladder

Start with a **bare repository** and unglamorous tooling: a dependency extractor, Tarjan’s algorithm, matrix powers, tree-sitter, an embedding model. From that alone an agent can already estimate a great deal. It can compute the code cut and its candidate alternatives under all four recovery objectives, whose disagreement profile is itself a fingerprint. It can compute reachability density, decoupling level, core size, element roles, cycles, levelization, and Martin’s  $(I, A, D)$ . It can read boundary interaction types out of interface signatures and connector idioms, along with binding times, declared visibility, and API surfaces. And it can infer technology bindings per module, naming-convention stereotypes, concern vectors from identifiers and comments, and the dominant-decomposition choice, meaning what the tree is organized by. One methodological caveat travels with all of it: extraction quality changes results as much as algorithm choice [87].

From **git history**: churn and hotspots; logical coupling; modularity violations, the highest-value single derived signal [28]; interface instability; ownership and  $\kappa_{OC}$ ; and drift dating, meaning when a divergence appeared and in which commit, which often links to *why*. From **runtime traces**: the realized interaction graph as against the potential one; hot paths; dynamic feature location. Requiring **human intent**, irreducibly but cheaply: the intended module map and allowed-dependency relation, which are the reflexion inputs [83], the fitness priorities, and rationale for tolerated divergences.

Of the seventeen basis entries, roughly nine are estimable from the bare repo, four more fall out of history, and only the intent-carriers need a person, via the draft-correct-enforce loop that reflexion proved and that language models make cheap.

### 8.2 The gap, and the tell

Against this, the composite working representation of current agents, across the major coding assistants and open scaffolds of 2024 through 2026, is: the file tree, a symbol and reference graph, lexical and embedding retrieval, and an optional human-written prose file of guidance [95, 96]. Hold that up against the basis. No intended-module map. No allowed-dependency relation. No concern vectors. No propagation structure. No history aggregation. No roles. No commitment calculus. No value weights. The agents hold the observables and none of the latents.

The prose context file is the tell, and it deserves to be named as such. Teams hand-maintain `CLAUDE.md` and its cousins because they know the latents matter. The file is a human-powered

cache of machine-computable quantities, lossy and stale by construction. It is the architecture-description pathology, dual maintenance, no incremental payoff, semantic distance from code, that killed the ADLs [93], replayed in miniature. The survivors of that extinction event are instructive. Architecture description persisted only where it became *derived from* or *enforced upon* the code: build-system visibility rules, typed interface schemas, conformance tests, deployment manifests [94]. The design implication for an agent is direct. Represent the basis as derived, continuously refreshed quantities plus a small store of confirmed intent, and let no fact live only in prose.

### 8.3 Why the decision store becomes viable now

The commitment calculus (§4.3) requires the one thing history never had: a populated decision store. Rationale capture failed for four documented, mutually reinforcing human-economic reasons. The capturer is not the beneficiary [57]. Formalization carries cognitive overhead and demands premature structure [58]. Capture intrudes on design flow mid-flight. And practitioners perceive the value as deferred and diffuse [59]. Notice what these four have in common. Every one is an asymmetry of human cost, not a defect of the representations. And every one dissolves for an agent that makes decisions itself, records them as a zero-marginal-cost side effect at the moment of deciding, infers structure rather than demanding it, and is its own future consumer. The mined reality that most real decisions live in issues, pull requests, and chat rather than in decision records [61] becomes, for such an agent, a recoverable backlog rather than a loss. And the one strong empirical result from the LLM-generation literature, that a recency window of three to five prior decisions from the same project is the best conditioning signal for the next one [97], is exactly what the trajectory theory predicts. The signal is in the path, and current representations carry it only by accident.

## 9 Honest limits and open problems

Four tensions survive this synthesis undischarged. We state them as constraints on the theory's ambitions, and two of them convert into its sharpest research questions.

**The fixed basis versus co-evolution.** The design-cognition record is unambiguous. Problem and solution spaces co-evolve [74]. Framing chooses what counts as a dimension [73]. Shaw and Brooks insist the space evolves mid-design [41, 10]. Our reconciliation, a fixed superset of generators with a time-varying attention weighting, is defensible. The basis variables recur across sixty years and eighteen systems, and that stability is evidence. But it is a hypothesis, not a theorem. Its falsifiable content is this: protocol studies of architects should find reframings that re-weight the basis, and should not find recurring decision dimensions that resist expression in it. A reframing that genuinely manufactures a new generator would falsify the fixed-superset claim.

**The identification problem.** Interaction minimization and change localization are different objectives that coincide only if change propagates uniformly along interaction edges (§3). No one has measured the actual kernel, the conditional probability that a change at  $u$  implicates  $v$ , as a function of the edge types and orders between them. The co-change record makes it measurable for the first time at scale. This is the paper's most concrete open empirical problem, and its answer calibrates every cut-quality metric in use.

**The metric's empirical status.** Claim 2 is theory-consistent. The asymmetry matches Tversky, and the successor-representation structure matches the navigation literature. But it

is unvalidated for architecture. No one has collected architects' similarity and substitutability judgments at scale and tested whether they embed with the predicted asymmetries, and whether generalization decays with the embedded distance [68]. Until that experiment is run, the metric is the best-motivated candidate, not a measured fact.

**Convexity at the edges.** Family-resemblance styles, disjunctive categories, and the hybrid discrete-continuous structure mean the Voronoi picture applies cleanly only region by region. We have treated convexity as a pressure shaping nameable vocabulary, which is the defensible reading [71, 70]. But it demotes one of the theory's prettiest formal properties to a tendency, and we would rather say so plainly than pretend otherwise.

## 10 Falsifiable predictions

**Prediction 1.** *Asymmetric similarity.* Architects' pairwise judgments of system similarity and migration difficulty will be systematically asymmetric, with the asymmetry oriented by re-decision cost, meaning dependents accrued and binding time, not by surface feature counts.

**Prediction 2.** *The congruence terms dominate.* In matched samples,  $\kappa_{OC}$  and the logical-physical congruence  $\kappa_{CR}$  will predict maintenance outcomes better than any single-graph structural metric, including propagation cost, extending the Vista result [22] from defects to velocity.

**Prediction 3.** *The two-way/one-way partition is stable.* New industrial migrations will continue to move bidirectionally on the coordinate axes, granularity, centralization, binding, concurrency model, state representation, and unidirectionally on the potentials, cyclic core, scattering, hub overload. An intentional, force-justified migration *toward* a larger cyclic core would count against the theory.

**Prediction 4.** *Recovery disagreement is lawful.* The disagreement profile among recovery objectives, density, idiom, concern, information, on a given system is reproducible and predicts which conformance rules its maintainers will accept, because both are functions of the same latent position.

**Prediction 5.** *An agent with the basis beats an agent with the observables.* On repository-level change tasks, an agent whose context is the estimated basis, meaning the intended map, the allowed-dependency relation, propagation structure, hotspots, roles, concern vectors, and the recent decision trajectory, will outperform an agent with the file tree, symbol graph, and embeddings at equal token budget. The margin will be largest on tasks whose blast radius crosses module boundaries.

**Prediction 6.** *Decision-trajectory signal.* Conditioning next-decision prediction on the typed decision graph, its states, edges, and restriction force, will beat the flat recency window of [97], because the window is a degenerate projection of the graph.

## 11 Coda: building the space

The construction recipe follows the theory. Coordinates: place every code unit by its structural address, graph position, role, concern vector, and binding profile, and place every decision by its commitment calculus. Locality: neighborhoods from dependency, call, co-change, and semantic distance, with betweenness only where the sub-space is continuous. Prototypes: the styles, patterns, and role archetypes as landmarks, and a repository's own core abstractions and canonical instances as its local landmarks. Operations: the six operators, the tactics, and the region jumps, exposed as the only mutation vocabulary. Metric: seeded by structure, reshaped



by realized co-navigation, asymmetric by construction. Value field: the project's criteria weights, held separate, orienting navigation without deforming terrain.

An agent so equipped does not read a codebase the way a search engine reads a corpus. It knows where it is, what is near, which moves are legal, what each move costs, and which way is up. That, and not a longer context window, and not a better prose file, is what it means for a machine to understand architecture.

The deeper wager of this paper is methodological. The field's sixty-year literature, read as a single corpus, is not a pile of opinions. It is a set of independent measurements of one underlying object, taken with different instruments. When the measurements converge, and we have shown that they converge, from Parnas's decision trees to Baldwin's options to boxology's tables to the migration post-mortems, the object is real. The latent variables of software architecture are not a metaphor awaiting rigor. They are the oldest finding in the field, waiting to be assembled.

## References

- [1] Parnas, D.L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM* 15(12):1053–1058.
- [2] Parnas, D.L. (1976). On the design and development of program families. *IEEE Transactions on Software Engineering* SE-2(1):1–9.
- [3] Parnas, D.L. (1979). Designing software for ease of extension and contraction. *IEEE TSE* SE-5(2):128–138.
- [4] Parnas, D.L., Clements, P.C., Weiss, D.M. (1985). The modular structure of complex systems. *IEEE TSE* SE-11(3):259–266.
- [5] Dijkstra, E.W. (1968). The structure of the “THE”-multiprogramming system. *Communications of the ACM* 11(5):341–346.
- [6] Dijkstra, E.W. (1974). On the role of scientific thought. EWD 447.
- [7] Simon, H.A. (1962). The architecture of complexity. *Proceedings of the American Philosophical Society* 106(6):467–482.
- [8] Simon, H.A., Ando, A. (1961). Aggregation of variables in dynamic systems. *Econometrica* 29:111–138.
- [9] Conway, M.E. (1968). How do committees invent? *Datamation* 14(4):28–31.
- [10] Brooks, F.P. (2010). *The Design of Design*. Addison-Wesley.
- [11] Stevens, W.P., Myers, G.J., Constantine, L.L. (1974). Structured design. *IBM Systems Journal* 13(2):115–139.
- [12] Lehman, M.M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE* 68(9):1060–1076.
- [13] Alexander, C. (1964). *Notes on the Synthesis of Form*. Harvard University Press.
- [14] Alexander, C., et al. (1977). *A Pattern Language*. Oxford University Press.
- [15] Baldwin, C.Y., Clark, K.B. (2000). *Design Rules, Vol. 1: The Power of Modularity*. MIT Press.
- [16] Baldwin, C.Y. (2008). Where do transactions come from? Modularity, transactions, and the boundaries of firms. *Industrial and Corporate Change* 17(1):155–195.
- [17] Colfer, L.J., Baldwin, C.Y. (2016). The mirroring hypothesis: theory, evidence, and exceptions. *Industrial and Corporate Change* 25(5):709–738.

- [18] Steward, D.V. (1981). The design structure system. *IEEE Transactions on Engineering Management* EM-28(3):71–74.
- [19] MacCormack, A., Rusnak, J., Baldwin, C.Y. (2006). Exploring the structure of complex software designs: an empirical study of open source and proprietary code. *Management Science* 52(7):1015–1030.
- [20] MacCormack, A., Baldwin, C.Y., Rusnak, J. (2012). Exploring the duality between product and organizational architectures: a test of the “mirroring” hypothesis. *Research Policy* 41(8):1309–1324.
- [21] Baldwin, C.Y., MacCormack, A., Rusnak, J. (2014). Hidden structure: using network methods to map system architecture. *Research Policy* 43(8):1381–1397.
- [22] Nagappan, N., Murphy, B., Basili, V. (2008). The influence of organizational structure on software quality. *Proc. ICSE 2008*.
- [23] Cataldo, M., Herbsleb, J.D., Carley, K.M. (2008). Socio-technical congruence. *Proc. ESEM 2008*.
- [24] Bird, C., Nagappan, N., Murphy, B., Gall, H., Devanbu, P. (2011). Don’t touch my code! Examining the effects of ownership on software quality. *Proc. ESEC/FSE 2011*.
- [25] Forsgren, N., Humble, J., Kim, G. (2018). *Accelerate*. IT Revolution Press.
- [26] Sullivan, K.J., Griswold, W.G., Cai, Y., Hallen, B. (2001). The structure and value of modularity in software design. *Proc. ESEC/FSE 2001*.
- [27] Cai, Y., Sullivan, K.J. (2006). Modularity analysis of logical design models. *Proc. ASE 2006*.
- [28] Mo, R., Cai, Y., Kazman, R., Xiao, L., Feng, Q. (2016). Decoupling level: a new metric for architectural maintenance complexity. *Proc. ICSE 2016*.
- [29] Lakos, J. (1996). *Large-Scale C++ Software Design*. Addison-Wesley.
- [30] Martin, R.C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- [31] Allen, E.B., Khoshgoftaar, T.M. (1999). Measuring coupling and cohesion: an information-theory approach. *Proc. IEEE METRICS 1999*.
- [32] Briand, L.C., Daly, J.W., Wüst, J. (1998–99). A unified framework for coupling/cohesion measurement in object-oriented systems. *IEEE TSE* 25(1) / *ESE* 3(1).
- [33] Mancoridis, S., Mitchell, B.S., et al. (1999). Bunch: a clustering tool for the recovery and maintenance of software system structures. *Proc. ICSM 1999*.
- [34] Andritsos, P., Tzerpos, V. (2005). Information-theoretic software clustering. *IEEE TSE* 31(2):150–165.
- [35] de Alfaro, L., Henzinger, T.A. (2001). Interface automata. *Proc. ESEC/FSE 2001*.
- [36] Benveniste, A., Caillaud, B., et al. (2018). Contracts for system design. *Foundations and Trends in EDA* 12(2–3).
- [37] Perry, D.E., Wolf, A.L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT SEN* 17(4):40–52.
- [38] Shaw, M., Garlan, D. (1996). *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall.
- [39] Shaw, M., Clements, P. (1997). A field guide to boxology: preliminary classification of architectural styles for software systems. *Proc. COMPSAC 1997*.
- [40] Lane, T.G. (1990). Studying software architecture through design spaces and rules. CMU/SEI-90-TR-18; and CMU/SEI-90-TR-22.

- [41] Shaw, M. (2012). The role of design spaces. *IEEE Software* 29(1):46–50.
- [42] Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD dissertation, UC Irvine.
- [43] Bass, L., Clements, P., Kazman, R. (2012). *Software Architecture in Practice*, 3rd ed. Addison-Wesley.
- [44] Kazman, R., Klein, M., Clements, P. (2000). ATAM: method for architecture evaluation. CMU/SEI-2000-TR-004.
- [45] ISO/IEC 25010:2011. Systems and software quality models.
- [46] Taylor, R.N., Medvidovic, N., Dashofy, E. (2010). *Software Architecture: Foundations, Theory, and Practice*. Wiley.
- [47] Mehta, N.R., Medvidovic, N., Phadke, S. (2000). Towards a taxonomy of software connectors. *Proc. ICSE 2000*.
- [48] Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software* 12(6):42–50.
- [49] Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley.
- [50] Cockburn, A. (2005). Hexagonal architecture (ports and adapters).
- [51] Fowler, M., Lewis, J. (2014). Microservices. martinofowler.com.
- [52] Newman, S. (2015). *Building Microservices*. O'Reilly.
- [53] Jansen, A., Bosch, J. (2005). Software architecture as a set of architectural design decisions. *Proc. WICSA 2005*.
- [54] van Gurp, J., Bosch, J. (2002). Design erosion: problems and causes. *Journal of Systems and Software* 61(2):105–119.
- [55] Kruchten, P. (2004). An ontology of architectural design decisions in software-intensive systems. *Proc. 2nd Groningen Workshop on Software Variability*.
- [56] MacLean, A., Young, R.M., Bellotti, V., Moran, T.P. (1991). Questions, options, and criteria: elements of design space analysis. *Human-Computer Interaction* 6(3–4):201–250.
- [57] Grudin, J. (1996). Evaluating opportunities for design capture. In Moran & Carroll (eds.), *Design Rationale*. Lawrence Erlbaum.
- [58] Shipman, F.M., Marshall, C.C. (1999). Formality considered harmful. *Computer Supported Cooperative Work* 8:333–352.
- [59] Tang, A., Babar, M.A., Gorton, I., Han, J. (2006). A survey of architecture design rationale. *Journal of Systems and Software* 79(12):1792–1804.
- [60] Buchgeher, G., et al. (2023). Using architecture decision records in open source projects — an MSR study on GitHub. *IEEE Software*.
- [61] Bhat, M., Shumaiev, K., et al. (2017). Automatic extraction of design decisions from issue management systems. arXiv:1704.04798.
- [62] Curtis, B., Krasner, H., Iscoe, N. (1988). A field study of the software design process for large systems. *Communications of the ACM* 31(11):1268–1287.
- [63] Zannier, C., Chiasson, M., Maurer, F. (2007). A model of design decision making based on empirical results of interviews with software designers. *Information and Software Technology* 49(6):637–653.
- [64] van Vliet, H., Tang, A. (2016). Decision making in software architecture. *Journal of Systems and Software* 117:638–644.
- [65] Fowler, M. (2003). Design — who needs an architect? *IEEE Software* 20(5):11–13.

- [66] Bezos, J. (2015). Amazon shareholder letter (Type 1 / Type 2 decisions).
- [67] Gärdénfors, P. (2000). *Conceptual Spaces: The Geometry of Thought*. MIT Press.
- [68] Shepard, R.N. (1987). Toward a universal law of generalization for psychological science. *Science* 237:1317–1323.
- [69] Tversky, A. (1977). Features of similarity. *Psychological Review* 84(4):327–352.
- [70] Hernández-Conde, J.V. (2017). A case against convexity in conceptual spaces. *Synthese* 194:4011–4037.
- [71] Jäger, G. (2007). The evolution of convex categories. *Linguistics and Philosophy* 30:551–564.
- [72] Goel, V., Pirolli, P. (1992). The structure of design problem spaces. *Cognitive Science* 16:395–429.
- [73] Schön, D.A. (1983). *The Reflective Practitioner*. Basic Books.
- [74] Dorst, K., Cross, N. (2001). Creativity in the design process: co-evolution of problem–solution. *Design Studies* 22(5):425–437.
- [75] Gero, J.S., Kannengiesser, U. (2004). The situated function–behaviour–structure framework. *Design Studies* 25(4):373–391.
- [76] Rosch, E. (1978). Principles of categorization. In *Cognition and Categorization*. Erlbaum.
- [77] Soloway, E., Ehrlich, K. (1984). Empirical studies of programming knowledge. *IEEE TSE* SE-10(5):595–609.
- [78] LaToza, T.D., Myers, B.A. (2010). Hard-to-answer questions about code. *Proc. PLATEAU 2010*; and Ko, A.J., DeLine, R., Venolia, G. (2007). Information needs in collocated software development teams. *Proc. ICSE 2007*.
- [79] Klein, G. (1998). *Sources of Power: How People Make Decisions*. MIT Press.
- [80] Dayan, P. (1993). Improving generalization for temporal difference learning: the successor representation. *Neural Computation* 5(4):613–624.
- [81] Stachenfeld, K.L., Botvinick, M.M., Gershman, S.J. (2017). The hippocampus as a predictive map. *Nature Neuroscience* 20:1643–1653.
- [82] Behrens, T.E.J., et al. (2018). What is a cognitive map? Organizing knowledge for flexible behavior. *Neuron* 100(2):490–509.
- [83] Murphy, G.C., Notkin, D., Sullivan, K. (1995). Software reflexion models: bridging the gap between source and high-level models. *Proc. FSE 1995*.
- [84] Tzerpos, V., Holt, R.C. (2000). ACDC: an algorithm for comprehension-driven clustering. *Proc. WCRE 2000*.
- [85] Garcia, J., Popescu, D., Mattmann, C., Medvidovic, N. (2011). Enhancing architectural recovery using concerns. *Proc. ASE 2011*.
- [86] Garcia, J., Ivkovic, I., Medvidovic, N. (2013). A comparative analysis of software architecture recovery techniques. *Proc. ASE 2013*.
- [87] Lutellier, T., et al. (2015). Comparing software architecture recovery techniques using accurate dependencies. *Proc. ICSE 2015*.
- [88] Louridas, P., Spinellis, D., Vlachos, V. (2008). Power laws in software. *ACM TOSEM* 18(1).
- [89] Tarr, P., Ossher, H., Harrison, W., Sutton, S.M. (1999). N degrees of separation: multi-dimensional separation of concerns. *Proc. ICSE 1999*.
- [90] Eaddy, M., et al. (2008). Do crosscutting concerns cause defects? *IEEE TSE* 34(4):497–515.

- [91] Behnamghader, P., et al. (2017). A large-scale study of architectural changes in open-source software systems. *Empirical Software Engineering* 22:1146–1193.
- [92] Azadi, U., Fontana, F.A., Taibi, D. (2019). Architectural smells detected by tools: a catalogue proposal. *Proc. TechDebt 2019*.
- [93] Malavolta, I., Lago, P., Muccini, H., Pelliccione, P., Tang, A. (2013). What industry needs from architectural languages. *IEEE TSE* 39(6):869–891.
- [94] Ford, N., Parsons, R., Kua, P. (2017). *Building Evolutionary Architectures*. O'Reilly.
- [95] Gauthier, P. (2023). Building a better repository map with tree-sitter. *aider.chat*.
- [96] Ouyang, S., et al. (2024). RepoGraph: enhancing AI software engineering with repository-level code graphs. *arXiv:2410.14684*.
- [97] Gupta, A., Dhar, R., Feitosa, D., Vaidhyathan, K. (2026). Context matters: evaluating context strategies for automated ADR generation using LLMs. *arXiv:2604.03826*.
- [98] Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns*. Addison-Wesley.
- [99] Tanenbaum, A., Torvalds, L. (1992). LINUX is obsolete (comp.os.minix debate). In *Open Sources* (O'Reilly, 1999), appendix.
- [100] Reis, C., Gribble, S.D. (2009). Isolating web programs in modern browser architectures. *Proc. EuroSys 2009*.
- [101] Reis, C., Moshchuk, A., Oskov, N. (2019). Site isolation: process separation for web sites within the browser. *Proc. USENIX Security 2019*.
- [102] Ahmeti, B., Linder, M., Groner, R., Wohlrab, R. (2024). Architecture decision records in practice: an action research study. *Proc. ECSA 2024*, LNCS 14889:333–349.
- [103] Yourdon, E., Constantine, L.L. (1979). *Structured Design*. Prentice Hall; and Myers, G.J. (1975). *Reliable Software through Composite Design*. Petrocelli/Charter.
- [104] Hipp, D.R. Architecture of SQLite. [sqlite.org/arch.html](https://sqlite.org/arch.html); and Gaffney, K.P., Prammer, M., Brasfield, L., Hipp, D.R., Kennedy, D., Patel, J.M. (2022). SQLite: past, present, and future. *Proceedings of the VLDB Endowment* 15(12):3535–3547.
- [105] Clark, A. (acdlite) (2016). React fiber architecture. [github.com/acdlite/react-fiber-architecture](https://github.com/acdlite/react-fiber-architecture).
- [106] Poettering, L. (2010). Rethinking PID 1. *0pointer.de*.
- [107] Kolny, M. (2023). Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%. Prime Video Tech Blog.
- [108] Noonan, A. (2018). Goodbye microservices. Segment Engineering Blog.
- [109] Westeinde, K. (2019). Deconstructing the monolith. Shopify Engineering.
- [110] Gluck, A. (2020). Introducing domain-oriented microservice architecture. Uber Engineering.
- [111] Istio team (2020). Introducing istiod: simplifying the control plane. *istio.io*.
- [112] Luo, S., et al. (2021). Characterizing microservice dependency and performance: Alibaba trace analysis. *Proc. SoCC 2021*.
- [113] Alexeev, A. nginx. In *The Architecture of Open Source Applications*, vol. II. [aosabook.org](https://aosabook.org).
- [114] Potter, S. Git. In *AOSA*, vol. II. [aosabook.org](https://aosabook.org).
- [115] Moir, K. Eclipse. In *AOSA*, vol. I. [aosabook.org](https://aosabook.org).
- [116] Sústrik, M. ZeroMQ. In *AOSA*, vol. II. [aosabook.org](https://aosabook.org).

- [117] Hunt, P., Konar, M., Junqueira, F.P., Reed, B. (2010). ZooKeeper: wait-free coordination for internet-scale systems. *Proc. USENIX ATC 2010*.
- [118] Brown, S. The C4 model for visualising software architecture. [c4model.com](http://c4model.com).
- [119] Kopp, O., Armbruster, A., Zimmermann, O. (2018). Markdown architectural decision records: format and tool support. *Proc. ZEUS 2018*, CEUR-WS vol. 2072.
- [120] Linnakangas, H. (2023). Let's make PostgreSQL multi-threaded. [pgsql-hackers mailing list](https://pgsql-hackers mailing list), [postgresql.org](https://postgresql.org).